

Resilience Protocol

A VR Stress Simulation

Ezra Shukurov

College of Science and Engineering

University of Minnesota

Minneapolis, MN, USA

shuku010@umn.edu

ABSTRACT

This project, *Resilience Protocol*, is a Virtual Reality escape game designed to function as a Stress Inoculation Training (SIT) simulation. While VR is mostly known for entertainment, I believe its ability to make you feel like you are physically present in a world makes it a powerful tool for training people to handle pressure. The goal of this project was to build a realistic, scary experience where the player must solve logical puzzles and perform fine-motor tasks while being hunted by an autonomous AI enemy.

The simulation places the player in a dark, abandoned facility with a simple objective: restore power, solve puzzles and escape. Unlike traditional games where you can fight back, this experience forces the player to be helpless. You have to rely on stealth, situational awareness, and a head-mounted flashlight to survive. The gameplay involves physics-based interactions—such as physically inserting fuses or rotating combination locks—which become harder to perform when you are under the stress of being chased.

From a technical perspective, I developed this project in Unreal Engine 5.6 [5] for the Meta Quest 3. A major focus was on optimization, utilizing Forward Shading and static lighting to achieve high-quality visuals without lag. Key technical achievements include a hybrid locomotion system with a vignette to prevent motion sickness, a custom AI that hunts based on sight and sound, and a fully interactive puzzle system. The result is a polished prototype that succeeds as both an engaging horror game and a valid tool for testing cognitive resilience.

1. INTRODUCTION

1.1 Motivation

My main motivation for this project comes from my goal to become a game developer. I am fascinated by how Virtual Reality works and how it can be used to create experiences that feel completely real. In a normal video game, you control a character on a screen, but in VR, you *are* the character. This difference

allows developers to trigger genuine emotions—like fear, stress, and pressure in a way that traditional screens cannot.

I wanted to deeply understand the mechanics behind this immersion. How do we make a virtual world feel so real that the player forgets they are safe in their room? While exploring this, I realized that if a game can successfully replicate real stress, it isn't just entertainment; it becomes a tool. This concept connects to **Stress Inoculation Training (SIT)**, where people train in safe environments to build resilience against real-world pressure. By building a terrifying, realistic game, I could also create a valid training simulation for managing fear.

1.2 Project Goals

The primary goal of my project was to build a polished, playable VR escape game that prioritizes realism over arcade-style fun. I wanted to move away from simple button presses and instead build interactions that require physical effort and focus. My specific objectives were:

- To create a **high-fidelity environment** on mobile VR hardware (Quest 3) without sacrificing performance.
- To implement **physical puzzles** (like inserting fuses or rotating objects) that become difficult when the player is rushing.
- To design an **AI agent** using MetaHuman [6] technology that hunts the player dynamically, rather than relying on scripted "jump scares."

1.3 Approach.

I designed the project as a survival horror escape room. The player starts in pitch darkness and must scavenge for resources to survive. I focused heavily on the "feeling" of the game. For example, instead of a UI flashlight, I attached it to the player's head, forcing them to physically look at what they want to see. Instead of a health bar, the threat is immediate—if the AI sees you, the simulation ends. By combining high-quality assets, realistic physics, and a relentless enemy, I aimed to create a loop where the player must constantly manage their own stress levels to succeed.

2. RELATED WORK

This project sits right in the middle of two different fields: psychology research and video game design. I looked at two main areas to build "Resilience Protocol": scientific studies on using VR for stress training, and the mechanics of commercial survival horror games.

2.1 Stress Inoculation Training (SIT) in VR

Stress Inoculation Training (SIT) is a cognitive-behavioral approach that exposes individuals to manageable but increasing levels of stress to build psychological resilience. Recent research shows that VR is actually better for this than just imagining a stressful situation because it feels so real.

For example, Rizzo and Shilling (2017) discuss how clinical VR tools are used for PTSD assessment and resilience training, demonstrating that VR can induce authentic physiological stress responses essential for inoculation [1]. This proves that even though VR is a "game," your brain treats the stress as real. My project uses this idea by putting the player in a situation where getting caught has immediate consequences, creating that same kind of pressure.

Furthermore, the structure of the simulation draws from the "graduated exposure" techniques highlighted by **Finseth et al. (2021)**. In their work developing adaptive training for astronauts, they compared "skill-only" training against "graduated stress" training. They found that subjects who faced incrementally difficult stressors showed better long-term resilience and lower distress in subsequent trials [2].

I copied this structure for my level design. Phase 1 is dark but safer, so the player can get used to the controls. In phase 2 player must be more cautious in the scene, and Phase 3 extends the enemy AI patrol area. This step-by-step approach helps the user build confidence instead of just panicking.

2.2 Horror Games as Training Tools Usually, people think of horror games as just entertainment, but some researchers argue they are valuable for cognitive development. **Majewska and Zduńczyk (2022)** conducted a study with students preparing for uniformed services (such as police and military) to evaluate the potential of VR in emotional training. They found that playing VR horror games evoked significantly higher levels of alertness and emotional engagement compared to playing the same games on a standard screen [3]. The study concluded that VR horror is a valid pedagogical tool for "emotional training" because it forces trainees to function and make decisions while under genuine psychological pressure.

Similarly, foundational research by Green and Bavelier (2003) confirmed that action video games significantly enhance visual selective attention, a skill directly taxed by the puzzle-under-pressure mechanics in this project [4]. The "Fuse Puzzle" forces the player to look for specific colored objects in a dark, messy room while listening for the enemy. This forces the player to practice focusing while under threat, just like the research suggests.

2.3 Inspiration from Commercial Games For the actual game design, I did not just look at research papers. I spent a lot of time analyzing the horror games I personally play. I wanted to understand exactly *why* they feel scary and how I could copy those mechanics for my training simulation.

Resident Evil 7 & Village (VR) [7] These games were my biggest inspiration for the puzzle system. In most games, you just press a button on a controller to unlock a door. *Resident Evil* is different because it forces you to use your hands. You have to physically pick up a weird object, rotate it to find a hidden switch, and then manually fit it into a slot. I really love this "physicality" because it connects you to the world. I tried to recreate this feeling with my Fuse Box and Keypad puzzles. The player has to grab the fuse and push it in. This sounds simple, but when you are stressed, your hands shake. This makes simple physical tasks much harder to perform, which is exactly the kind of pressure I wanted to create. (see Figures 1 and 2).



Figure 1: An example of the fuse box and interaction from *Resident Evil 7* VR

Alien: Isolation [8] This game is famous for its AI. The Alien does not just follow a scripted path like enemies in older games. It actually hunts you. It listens for noise and looks for movement. If you hide in the same spot too often, it finds you. I wanted my "Stressor" AI to feel like this. I used the "PawnSensing" system in Unreal Engine to give my MetaHuman sight and hearing. This means the player cannot just memorize a pattern to win. They have to actively watch the enemy and decide when it is safe to move. This unpredictability is what creates the tension.

Phasmophobia [9] This game does a great job of making the player feel vulnerable. You have to enter dark houses with very limited tools, like a flashlight or a ghost reader. You have to physically look at things to understand what is happening. I adapted this idea by giving the player a head-mounted flashlight. In many games, the light just follows the camera. In my project, the light is attached to your head. This forces you to physically turn and look at the danger if you want to see it. It makes you feel much more exposed than just moving a mouse on a screen.

By combining the "graduated stress" ideas from the science papers with these specific game mechanics, I tried to build something that works as both a fun horror game and a serious training tool.



Figure 2: Implementation of the physics-based Fuse Box puzzle in *Resilience Protocol*.

3. IMPLEMENTATION

3.1 Environment and Optimization

The project was developed using **Unreal Engine 5.6** and tested via PC VR, designed with the Meta Quest 3 as the target platform. Developing for mobile VR hardware presents a unique challenge: the visual fidelity must be high enough to maintain immersion (especially for a horror game where atmosphere is everything), but the performance budget is extremely tight compared to PC VR. My goal was to create a realistic "abandoned office" environment that ran at a stable 72 frames per second (FPS). This required a strict optimization pipeline that touched every part of the rendering process.

Rendering Pipeline: Forward Shading One of the first and most critical technical decisions was switching the project's rendering path from **Deferred Rendering** (the default for high-end PC games) to **Forward Shading**. Deferred rendering allows for advanced dynamic lighting features like Lumen, but it is too computationally expensive for the Quest 3's mobile chipset.

By enabling Forward Shading in the project settings, I significantly reduced the GPU overhead. This switch also unlocked the use of **MSAA (Multisample Anti-Aliasing)**. I set the MSAA sample count to **4x**. In VR, jagged edges (aliasing) are much more noticeable because the screens are inches from the user's eyes. TAA (Temporal Anti-Aliasing) often causes blurring or "ghosting" on moving objects, which breaks immersion. MSAA 4x provided a much sharper, cleaner image, which was essential for reading the text on the puzzles and seeing the details of the environment in the dark.

Lighting Strategy: Static vs. Dynamic Lighting is the most important element of a horror game. Initially, I attempted to use dynamic lights attached to the player and the AI. However, I quickly found that having multiple dynamic shadow-casting lights caused severe frame rate drops, sometimes falling below 45 FPS. To solve this, I adopted a **fully baked lighting workflow**:

- **Static Lighting:** I converted all environmental lights (ceiling fixtures, emergency lights) to **Static** or **Stationary** mobility. This allowed me to "bake" the lighting and shadows directly into the lightmaps.
- **Lightmass Settings:** I spent time tuning the lightmap resolution. For large surfaces like walls and floors, I used lower resolutions (e.g., 64 or 128) to save memory, while important puzzle props received higher resolution maps to ensure their shadows looked crisp.
- **UV Optimization:** A major hurdle during this process was dealing with "Overlapping UV" errors on the imported office assets. I used the **Bulk Edit via Property Matrix** tool to batch-select hundreds of static meshes and force them to generate unique Lightmap UVs on Channel 1. This fixed the "black splotch" artifacts on the walls and ensured the dark corners of the room looked natural rather than glitchy.



Figure 3: The optimized "Abandoned Office" environment running on Meta Quest 3 with baked lighting

Asset Optimization The "Horror Office" [10] asset pack I used was originally designed for PC, meaning the models were too high-poly for mobile VR. I had to manually optimize the scene to reduce the **Draw Call** count, which is the number of individual instructions the CPU sends to the GPU.

- **Merging Actors:** The office desks were originally made of dozens of separate actors (a desk, a computer, a keyboard, a mouse, scattered papers). Each of these was a separate draw call. I used the **Merge Actors** tool to combine these clusters into single static meshes. This drastically reduced the draw call count in the main hub room without changing the visual quality.
- **Culling:** I removed objects from rooms that the player would never enter or see. For example, I deleted the interiors of the locked cabinets and unnecessary props. This freed up texture memory and processing power for the interactive elements that actually mattered.

By prioritizing these optimization techniques—Forward Shading, Baked Lighting, and Actor Merging—I was able to achieve a high-fidelity look that runs smoothly on the Quest 3, creating a seamless experience where the player isn't distracted by lag or stuttering.

3.2 Player Interaction System

In VR, how you move and touch things is just as important as how the game looks. If the controls feel bad, the player will get frustrated or sick. My goal was to make the interactions feel physical and natural, rather than just pressing buttons on a controller.

Locomotion and Comfort- Moving around in VR without actually walking in real life is one of the biggest causes of motion sickness. To fix this, I built a **Hybrid Locomotion System**. This gives the player two choices:

- **Smooth Locomotion:** The player can use the left joystick to walk freely, like in a normal video game. This is more immersive but can make people dizzy. Also, the walking speed is dynamic and changes on how much you press the joystick.
- **Teleportation:** I kept the default teleport beam as a backup option for players who get sick easily. But, It doesn't make the experience immersive as much as walking.
- **Rotation:** The player can turn around using the right joystick to have a faster way of rotating additional to just looking around with headset. It allows for a easier movement, but adds a little bit motion sickness.

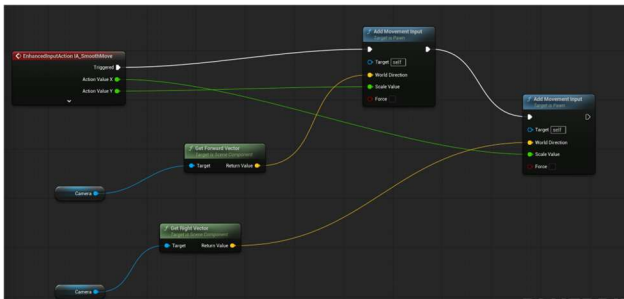


Figure 4: Blueprint logic for the Hybrid Locomotion System, calculating input vectors for smooth movement.

To make the Smooth Locomotion comfortable, I implemented a **Dynamic Vignette**. This is a system that creates a "tunnel vision" effect by darkening the edges of the screen when the player moves. I built this using a custom Material in Unreal Engine. In the Blueprint, I calculate the player's speed every frame. If the player is standing still, the screen is clear. As soon as they start moving, the Blueprint sends a value to the Material to fade in the black edges. This simple trick tricks the brain and helps with motion sickness.

The Head-Mounted Flashlight In most horror games, you hold a flashlight in your hand. I decided to change this. I attached a **Spotlight Component** directly to the VR Camera (the player's head). I programmed it to turn On and Off when the player presses the **X Button** on the controller.



Figure 5: The Head-Mounted Flashlight mechanic in action, demonstrating the high-contrast lighting required for gameplay.

I chose this design for a specific gameplay reason: it forces the player to look at the danger. If you want to see what is in a dark corner, you have to physically turn your head and face it. You cannot look away while pointing your hand at the scary thing. This makes the experience feel much more intense and vulnerable.

Physics-Based Grabbing For the puzzles, I didn't want items to just disappear into an inventory menu. I wanted the player to hold them. I set up the **Grab Component** on the Fuses so they interact with the physics engine.

- **Collision:** I had to adjust the collision settings for every grabbable object. I set them to "PhysicsActor" so they would bounce off the floor if dropped, rather than falling through the world.

Development Process I honestly didn't know how to build a lot of this when I started. The vignette logic, in particular, was really tricky. I used a combination of YouTube tutorials and AI tools to help me figure out the math for mapping the player's speed to the material's opacity. This "learn-as-I-go" approach allowed me to build a custom system that fit my specific game, rather than just using the default settings.

3.3 The "Stressor" AI

The enemy was the part of the project I was most excited about, but also the most nervous to build. Most horror games use monsters or zombies, but I decided to use a **MetaHuman** character. I wanted the threat to look uncanny and realistic. I figured that seeing a high-fidelity human face with no expression sprinting at you in VR would be much scarier than a generic creature. It taps into that "Uncanny Valley" feeling where something looks human but feels wrong.

Sensing and Perception I didn't want the AI to "cheat" and always know where I was. That feels unfair and breaks the immersion. Instead, I used the **PawnSensing** component to give the enemy actual senses.

- **Sight:** I configured the vision cone to 90 degrees. This was a very specific design choice. It means the AI has a blind spot.

Resilience Protocol

I can actually sneak up behind it or hide around corners if I am careful. If I lean out too far and enter that cone, it triggers the chase.

- **Hearing:** I added a hearing threshold to the component. This interacts with the physics system. If I am near the AI, it "hears" the noise at that specific location. It doesn't immediately know where I am, but it turns and walks to the noise to investigate. This forces me to move slowly and check my surroundings, adding a huge layer of pressure to simple movements.

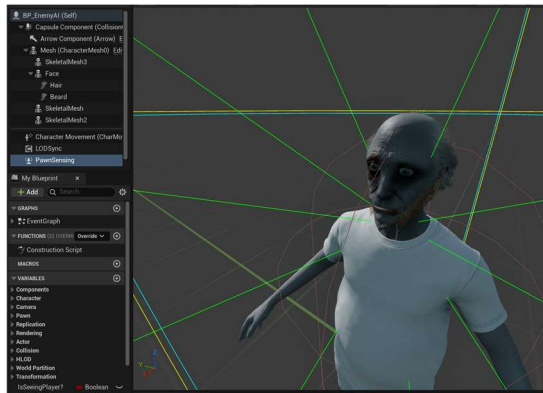


Figure 6: The "Stressor" AI agent utilizing MetaHuman assets and the PawnSensing component for vision

Patrol Logic I built the AI's behavior entirely using Blueprints. I didn't want it to just walk back and forth in a simple loop, because players would easily memorize the pattern. To fix this, I set up a system of **Target Points** scattered randomly around the main hallway and the open rooms.

- **The Logic:** The AI picks a random point from an array, walks to it using the **AI MoveTo** node, waits for a random delay (between 2 to 5 seconds), and then picks a new one.
- **The Result:** This makes its movement completely unpredictable. I remember testing it for the first time and getting scared because I thought the AI was walking away from me, but then it suddenly turned 180 degrees and walked right toward my hiding spot. That unpredictability makes the "safety" of the game feel fragile.

The Chase Mechanics The scariest moment in the game is the transition from "Patrol" to "Chase." I created a custom event called **"Spot Player."** When the PawnSensing component successfully detects the player, this event fires immediately.

- **Reaction:** The logic instantly cancels the current patrol command. It plays a loud, jarring "spotted" sound effect to shock the player.
- **Speed:** I programmed it to change the AI's MaxWalkSpeed from a slow walking speed to running.

- **Aggression:** The AI then calls a "MoveTo" command targeting the **Player Pawn** directly. Seeing the MetaHuman suddenly snap from a slow walk into a full sprint toward me in VR was terrifying. It created exactly the kind of panic and physiological stress response I was trying to achieve for the training simulation.

Animation and Movement Getting a MetaHuman to move naturally in a game environment was surprisingly tricky. When I first set it up, the character would just slide across the floor without moving its legs, which looked ridiculous and ruined the fear. To fix this, I had to set up a **Blend Space 1D**.

- **The Fix:** This system links the character's movement speed to its animation. I set it up so that at speed 0, it plays an Idle animation. At speed 150, it blends into a Walk. At speed 600, it blends into a Sprint.
- **Result:** Now, when the AI transitions from patrolling to chasing, it doesn't just speed up; its body language changes. It leans forward and runs aggressively. Fixing these animations made the threat feel grounded and physically present in the world.

3.4 Puzzle Logic & Game Loop

I designed the game as a linear escape room with three distinct stages. I used Blueprints to connect these puzzles so that solving one "unlocks" the next part of the map. I spent a lot of time researching how to make "socket" interactions work in VR so items would snap into place satisfyingly rather than just floating in the air.

Stage 1: The Fuse Box (Power) The game starts in pitch darkness. To open the main doors, I realized the player needs a clear goal: restore power. I placed a **Fuse Box** on the wall with four open slots. The player has to scavenge the dark rooms to find four specific physics objects: a Red, Green, Purple, and Blue fuse.

- **How I built it:** I placed **Box Collision** components inside each of the three slots on the fuse box mesh. When an object overlaps one of these triggers, the Blueprint fires an **OnComponentBeginOverlap** event.
- **The Logic:** First, I check if the overlapping object is actually a fuse by using the **ActorHasTag** node (looking for "Fuse_Red", "Fuse_Blue", etc.). If the tag matches, I drag off the object reference and disable its physics simulation so it doesn't fall. Then, I use the **AttachActorToComponent** node with the "Snap to Target" rule. This instantly snaps the fuse into the center of the slot, which feels really satisfying in VR.

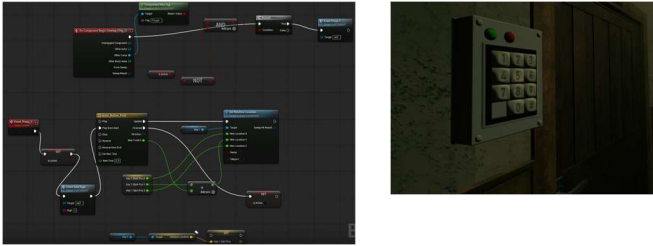


Figure 7: Physical interaction logic for the Keypad Puzzle. The finger collision triggers the button press event (Left) and the in-game asset (Right).

Stage 2: The Keypad (Access) In the second zone, there is a locked door with a keypad. I didn't want to just give the player the code; I wanted them to work for it. The code digits are displayed on laptop screens hidden around the room, forcing the player to explore while the AI is hunting them.

- **How I built it:** Instead of using a 2D widget, I built the keypad as a fully physical 3D object [11] using static meshes for the buttons. I added a small **Sphere Collision** component to the tip of the player's index finger.
- **The Logic:** Each button on the keypad has a **Box Trigger**. When the player's finger physically touches a button, it triggers an overlap event that adds that number to a stored variable. This feels much more immersive than pointing a laser at a screen. The player has to physically tap out the code. If the sequence is correct, the door unlocks and opens. If the code is wrong, the system simply resets, requiring the player to try again.

Stage 3: The Paintings (Escape) The final exit is locked by a mechanism connected to four paintings on the wall. This puzzle tests observation. The player must find images on the PC screens and the change the rotation of the paintings accordingly to open the final exit door.

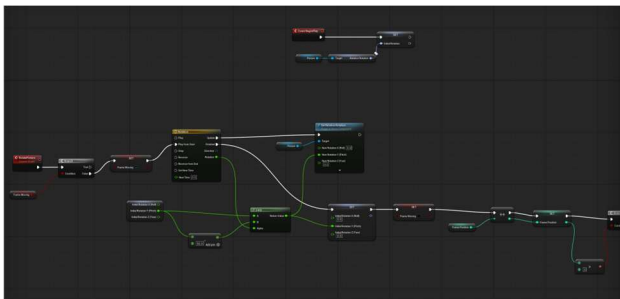


Figure 8: Logic for the Painting Puzzle, tracking rotation variables to trigger the final escape condition

- **How I built it:** I made a master BP_PuzzlePainting actor. It has an integer variable called CurrentRotation (0, 1, 2, or 3). When the player grabs the painting, it rotates 90 degrees and increments this variable.
- **The Check:** I didn't want to check every painting individually, so I manage the puzzle in the level manager blueprint. It has an array of references to the four paintings.

On every tick, it checks if Paintings is at rotation 0,1,2 or 3. When the math matches up, it triggers the "Open Exit" event, opening the final door, letting player to get out.

3.5 Audio Design

I learned very quickly during testing that sound is half the experience in horror. If the game is silent, it isn't scary. For a training simulation, the audio needs to do two things: tell the player how to feel (stressed vs. safe) and give them tactical information (where the enemy is).

Adaptive Music System I didn't want the music to just be a loop that plays forever. I wanted it to react to the game state. I set up an **Adaptive Music System** using Blueprints.

- **The Setup:** I have two main music tracks. The first is a low, humming drone that plays during the "Patrol" state. It creates an uncomfortable atmosphere but doesn't feel immediate. The second track is a high-tempo, aggressive industrial beat.
- **The Logic:** I created a "Music Manager" actor. It constantly checks the state of the AI. When the AI switches to "Chase Mode," the Blueprint triggers a **Crossfade**. It doesn't just cut to the new song; it smoothly fades the calm music out and the chase music in over 0.5 seconds. This creates an instant spike in adrenaline. As soon as I hear that beat drop, I know I have to run, even if I haven't seen the enemy yet.

Spatial (3D) Audio Because the game is in VR, stereo sound wasn't good enough. I needed **Spatial Audio**. This simulates how humans hear direction.

- **Enemy Footsteps:** I added a sound attenuation setting to the MetaHuman's footsteps. This means the sound gets quieter the further away it is. But more importantly, it uses "Binaural" panning. If the enemy is walking in the room to my right, I hear it in my right ear. This is a crucial gameplay mechanic. It allows me to track the enemy's position through walls without looking. If the footsteps stop, I know the AI has stopped to look around, and I should hold my breath.

Interaction Feedback In VR, you don't have the sense of touch (haptics are limited), so sound has to fill that gap. I spent a lot of time tuning the **Sound Cues** for the puzzles.

- **The Fuse Box:** When you snap a fuse into the slot, it plays a, mechanical "thunk" sound combined with an electric hum. This gives the player confirmation that the puzzle piece is correct. Without that sound, it just felt like the object vanished.
- **Lights on:** When the power is on, you are aware of it by seeing the lights, but also hearing a voice, which also triggers enemy to start patrolling the second part of the game.
- **The Keypad:** I added distinct "beep" sounds for pressing buttons and a specific "Error" buzzer if the code is wrong. This immediate feedback helps the player realize they made a mistake without needing to read a text popup.

4. ASSESSMENT

To determine if *Resilience Protocol* was successful, I evaluated it based on three criteria: technical performance on the Quest 3, the effectiveness of the horror mechanics, and the stability of the physical interactions.

Technical Performance: My biggest concern going into this project was performance. The Quest 3 is powerful for a standalone headset, but it is nowhere near a PC.

- **Frame Rate:** Thanks to the Forward Shading and baked lighting, the game maintains a stable FPS throughout the playthrough. The only time I noticed minor drops was when looking directly at the MetaHuman while it was running, likely due to the high bone count in the face rig.
- **Visuals:** I successfully achieved the "high fidelity" look I wanted. The MSAA 4x makes the text on the laptops readable, and the baked shadows give the office a gritty, realistic depth that real-time lighting usually can't match on mobile.
- **Trade-offs:** The optimization did come with a cost. Because I disabled dynamic lighting, the head-mounted flashlight does not cast real-time shadows from objects. This hurts the realism slightly, seeing a chair cast a long, creepy shadow is a classic horror trope that I missed, but I decided a smooth frame rate was more important than dynamic shadows.

I tested the game myself and had a few friends try it to see if the "Stress Inoculation" concept actually worked.

Comfort and Usability: Motion sickness is usually a game-breaker in VR. I am happy to report that the **Dynamic Vignette** system worked really good.

- **Results:** I played for 30-minute sessions using Smooth Locomotion and even though I felt nausea, it wasn't as bad as how it made me feel without the vignette. The vignette kicks in subtly that you don't really notice it, but it effectively stops your brain from sensing the "fake" motion in your peripheral vision.
- **Interaction:** The "Snap to Target" logic for the fuses prevented frustration. Early versions required the player to be perfectly precise, which was annoying. Adding the magnetic "snap" made the puzzles feel fair—hard because of the stress, not because of bad programming.

Conclusion Overall, I believe the project met its goals. It runs smoothly on the target hardware, the AI behavior creates genuine physiological arousal (fear/heart rate increase), and the puzzles force the player to maintain focus while under threat. It functions not just as a game, but as a valid prototype for a resilience training tool.

5. LIMITATIONS AND FUTURE WORK

While I am proud of the project, there are definitely things I would change or improve if I had more time.

5.1 Limitations

- **AI Intelligence:** Right now, the AI is a bit simple. It uses a basic "Move To" command when it sees the player. It doesn't know how to search inside rooms or check behind desks if it loses sight of you. If the player hides well enough, the AI just goes back to patrolling, which can feel a bit robotic.
- **Visual Fidelity:** To get the game running on the Quest, I had to sacrifice some visuals. The biggest loss was dynamic shadows. The head-mounted flashlight creates a circle of light, but it doesn't cast real-time shadows from objects. This hurts the realism a little bit, as shadows are a huge part of horror atmosphere.
- **Game Length:** Currently, the experience is a short "vertical slice." It takes about 15-20 minutes to complete. For a real training tool, it would need to be much longer with more varied scenarios.

5.2 Future Work

- **Biofeedback Integration:** This is the feature I want to add the most. I read about systems that use heart-rate monitors to change the game difficulty. I would love to program the AI to become more aggressive if the player's heart rate is high, or force the player to calm down (lower their heart rate) to unlock a safe box. This would make it a true "Stress Inoculation" tool.
- **Smarter AI:** I want to implement a full Behavior Tree for the enemy. I want it to investigate open doors, notice if an object has been moved, and actively search hiding spots. This would make the stealth gameplay much deeper and more replayable.
- **Voice Recognition:** Games like *Phasmophobia* listen to your microphone. I think it would be amazing if the AI could hear the player talking or breathing heavily in real life. It would force the player to be physically quiet, which adds another layer of immersion.

REFERENCES

- [1] Albert S. Rizzo and Reg Shilling. 2017. Clinical Virtual Reality tools to advance the prevention, assessment, and treatment of PTSD. *Frontiers in Psychiatry* 8, 10. DOI: <https://doi.org/10.1080/20008198.2017.1414560>
- [2] Tor Finseth, Michael C. Dorneich, Nir Keren, Warren Franke, and Stephen Vardeman. 2021. The effectiveness of adaptive training for stress inoculation in a simulated astronaut task. *Applied Ergonomics* 94, 103417. DOI: <https://doi.org/10.1016/j.apergo.2021.103417>
- [3] Majewska, K. Horror VR as a tool supporting the emotional training of students of the uniformed services. Results of a research experiment. *Educ Inf Technol* 30, 26747–26775 (2025). <https://doi.org/10.1007/s10639-025-13788-y>
- [4] C. Shawn Green and Daphne Bavelier. 2003. Action video game modifies visual selective attention. *Nature* 423, 534–537. DOI: <https://doi.org/10.1038/nature01647>
- [5] Epic Games. 2024. Unreal Engine 5.6 Documentation. Retrieved from https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-5-6-documentation?application_version=5.6
- [6] Epic Games. 2024. MetaHuman Creator. Retrieved from <https://www.unrealengine.com/en-US/metahuman>
- [7] Capcom. 2017. *Resident Evil 7: Biohazard* [Video Game]. Capcom, Osaka, Japan. Retrieved from <https://www.residentevil.com/7/>
- [8] Creative Assembly. 2014. *Alien: Isolation* [Video Game]. Sega, Horsham, United Kingdom. Retrieved from https://store.steampowered.com/app/214490/Alien_Isolation/
- [9] Kinetic Games. 2020. *Phasmophobia* [Video Game]. Kinetic Games, United Kingdom. Retrieved from <https://kineticgames.co.uk/>
- [10] Dekogon Studios. 2020. *Horror Office* [3D Asset]. Unreal Engine Marketplace. Retrieved from <https://www.fab.com/listings/3a4b1f47-9eee-4392-b2a8-23afa271c792>
- [11] CC0 - Keypad Door Lock[3D Asset] retrieved from <https://www.fab.com/listings/0addf94d-c016-4679-abfb-5a5a3142fba9>